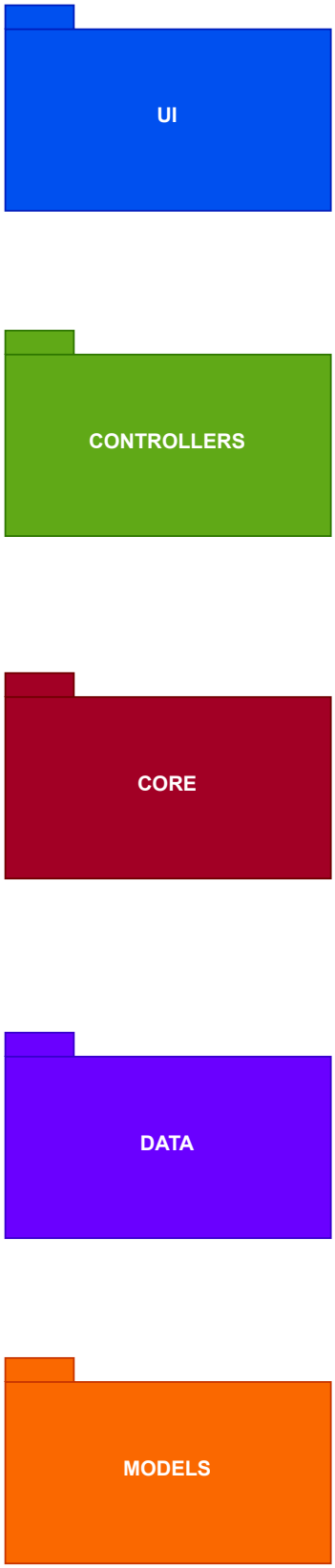
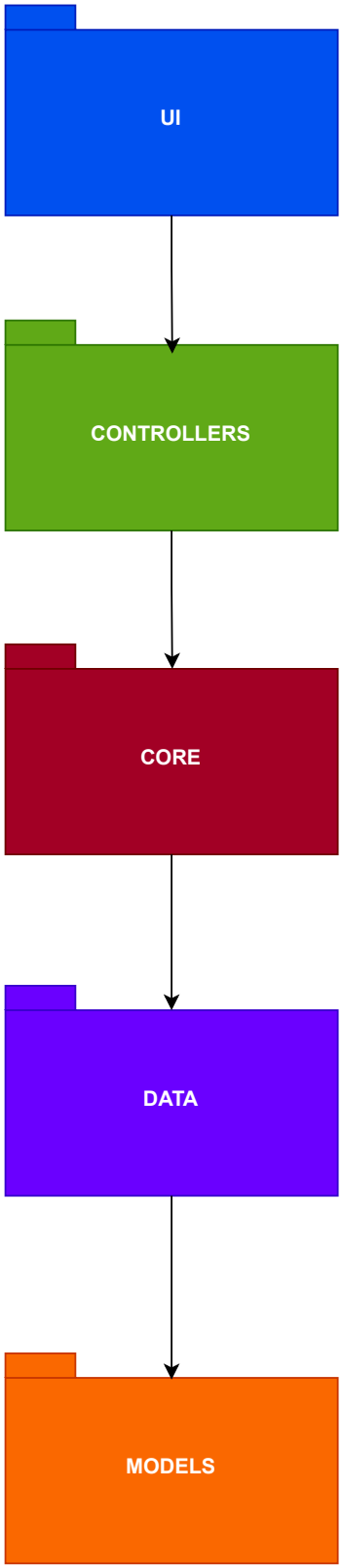
Det här diagrammet visar systemets lagerarkitektur och hur projektet är organiserat.



Alla beroenden går nedåt i lagerstrukturen, vilket följer SOLID och gör systemet lätt att underhålla.



UI-lagret ansvarar för all visuell presentation och användarinteraktion.



Controllers styr flödet i systemet och anropar Core-lagret



Core innehåller affärslogiken och är systemets hjärna.

InputService

+ ReadLineWithEsc() : string?
+ ReadString(string) : string?
+ ReadStringAllowEmpty(string): string?
+ ReadInt(string) : int?
+ ReadDecimal(string) : decimal?
+ ReadDate(string) : DateTime?
+ ReadUnit(string) : PriceUnit?
+ ShowError(string) : void

InputParser

+ Parse(string, List<Product>): (Product product, decimal quantity)?

CommandParser

+ IsPayCommand(string): bool
+ IsCancelCommand(string) : bool
+ RIsDeleteCommand(string): bool
+ ExtractPluFromDelete(string) : string?

CartService

+ AddItem(Cart, Product, int, decimal): void
+ AddItemByWeight(Cart, Product, decimal, decimal) : void
+ RemoveItem(Cart, string): bool
+ CalculateTotal(Cart) : decimal
+ GetItems(Cart): IEnumerable<CartItem>
+ Clear(Cart) : void

ProductService

+ AddProduct(List<Product): Product?
+ DeleteProduct(List<Product>, string?) : bool
+EditProduct(List<Product>, string?): bool

CampaignService

+ GetPrice(ProductI): decimal
+GetMultiBuyTotal(Product, decimal) : decimal
+ GetDiscount(Produc, decimal): decimal
+ GetCampaignName(Product) : string

DiscountService

+ CalculateDiscount(Product, decimal): decimal

PaymentService

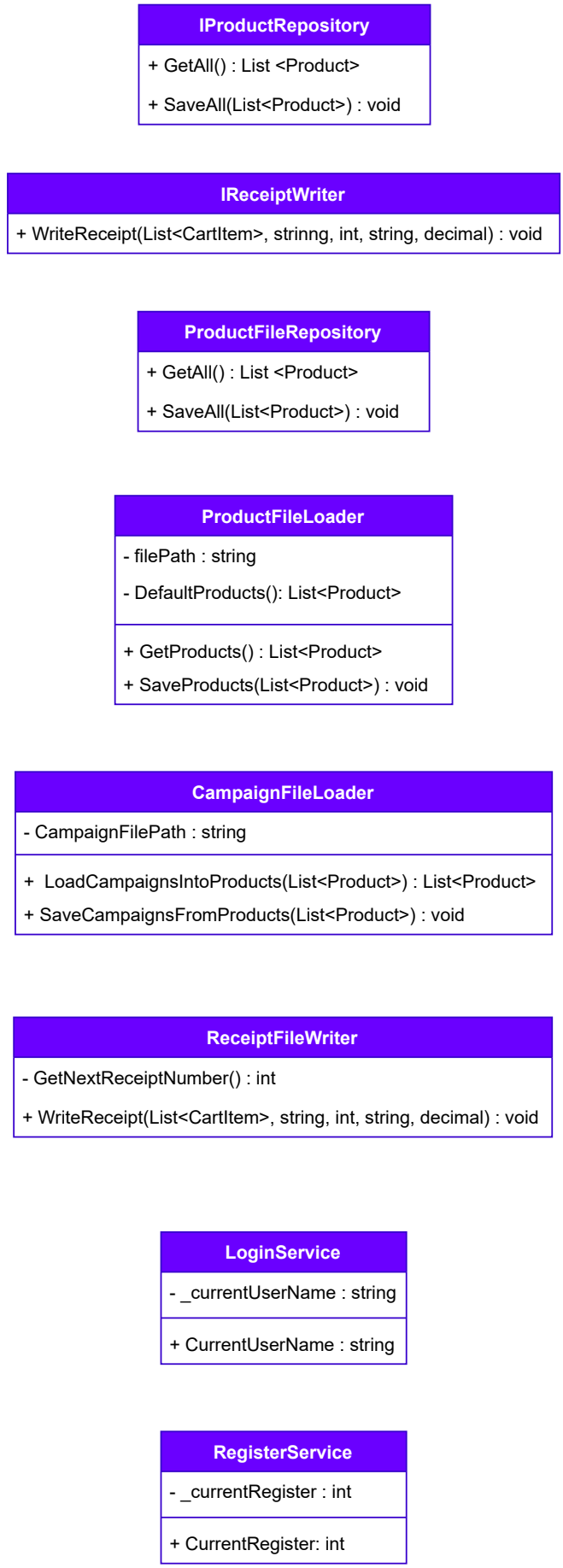
+ CompletePayment(Cart): bool
+ReadCashAmount() : decimal

ReceiptNumberService

- CounterFile: string

+ GetNext(): int
+ SaveCurrentReceiptNumber(int) : void

Data-lagret hanterar lagring, filåtkomst och persistens



Models representerar systemets dataobjekt

